Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут» ім. Ігоря Сікорського

Лабораторна робота №2

з дисципліни «Web-дизайн»

«Розробка функціональності Web-додатка мовою Javascript»

Виконав студент групи: КВ-33

Козлов Сергій

Київ 2026

Мета

Ознайомитись із засобами мови Javascript та навчитись їх застосовувати для побудови Web-інтерфейсу користувача.

Загальне завдання

Розробити функціональність для статичних сторінок Web-додатку першої лабораторної роботи із використанням шаблону MVC або unidirectional data flow.

Web-додаток має містити наступні сторінки:

- реєстрація користувача (поля: ім'я, email, стать, дата народження)
- вхід до сайту (поля: email, пароль)
- профіль користувача (поля у табличному вигляді)
- про додаток (емблема додатку, короткий опис додатка)
- робоча сторінка додатка (розробляється самостійно студентом відповідно до обраної тематики)

Тематика додатку

Блог з коментарями.

Завдання для обраної тематики

Реалізувати можливість введення (перегляду списку, додавання, вилучення) повідомлень блогу (назва, текстовий вміст), а також коментарів до кожного повідомлення.

Посилання

Оскільки Лабораторна робота №2 базується на Лабораторній роботі №1, вихідний код розміщений і тому ж репозиторії.

Репозиторій GitHub: https://github.com/Teollan/2026-wd-lab1

Демо додатку (GitHub Pages): https://teollan.github.io/2026-wd-lab1/

Контакт в Telegram: https://t.me/Teollan

Опис Web-додатка BlogSpace

Загальні відомості

BlogSpace — це Web-додаток, що реалізує блог-платформу з підтримкою коментарів.

Функціональність

Додаток містить наступні сторінки:

- **About** (Про додаток) – головна сторінка з логотипом, описом платформи та переліком її можливостей. Для неавторизованих користувачів відображається кнопка реєстрації, для авторизованих – перехід до стрічки публікацій.
- **Sign Up** (Реєстрація) – форма створення облікового запису користувача. Доступні поля: ім'я користувача, електронна пошта, пароль, стать та дата народження.
- **Sign In** (Вхід) – форма аутентифікації за допомогою електронної пошти та пароля.
- **Feed** (Стрічка) – сторінка перегляду публікацій інших користувачів на основі інтересів користувача. Кожна публікація містить ім'я автора, дату, заголовок, текст та секцію коментарів з можливістю залишити власний коментар.
- **My Posts** (Мої публікації) – сторінка з публікаціями поточного користувача. На відміну від загальної стрічки, публікацію можна відредагувати або видалити повністю.
- **New Post** (Нова публікація) – форма створення нової публікації з заголовком та текстовим вмістом.
- **Edit Post** (Відредагувати публікацію) – форма редагування існуючої публікації.
- **Profile** (Профіль) – сторінка профілю, де можна переглянути інформацію про поточного користувача. Кнопка “Log out” дозволяє вийти з акаунту.

Інструменти розробки

- HTML5 – розмітка сторінок
- Tailwind CSS v4 – стилізація інтерфейсу
- TypeScript – скрипти і типізація
- Vite – збірка проекту та локальний сервер розробки
- GitHub Pages – розгортання сайту

Unidirectional Data Flow і Flux-подібна архітектура

В основу реактивної системи застосунку покладено паттерн однонаправленого потоку даних (Unidirectional Data Flow), реалізований за принципами архітектури Flux. Центральним механізмом є клас Publisher, який забезпечує підписку на зміни через метод subscribe() та оповіщення підписників через notify().

Його наслідують два ключових класи:

- Dispatcher, що приймає дії користувача та розсилає їх сховищам стану
- Store, що зберігає стан модуля та оновлює його відповідно до дії.

Взаємодія користувача з інтерфейсом ініціює створення об'єкта Action (через HTML-атрибут data-action), який передається до Dispatcher. Диспетчер розсилає дію до відповідного Store, де обробник оновлює стан через setState(), що викликає notify() і перемальовує підписані ReactiveComponent. Таким чином формується замкнений цикл: View → Action → Dispatcher → Store → View, де дані завжди рухаються в одному напрямку, що спрощує відлагодження та забезпечує передбачуваність поведінки системи.

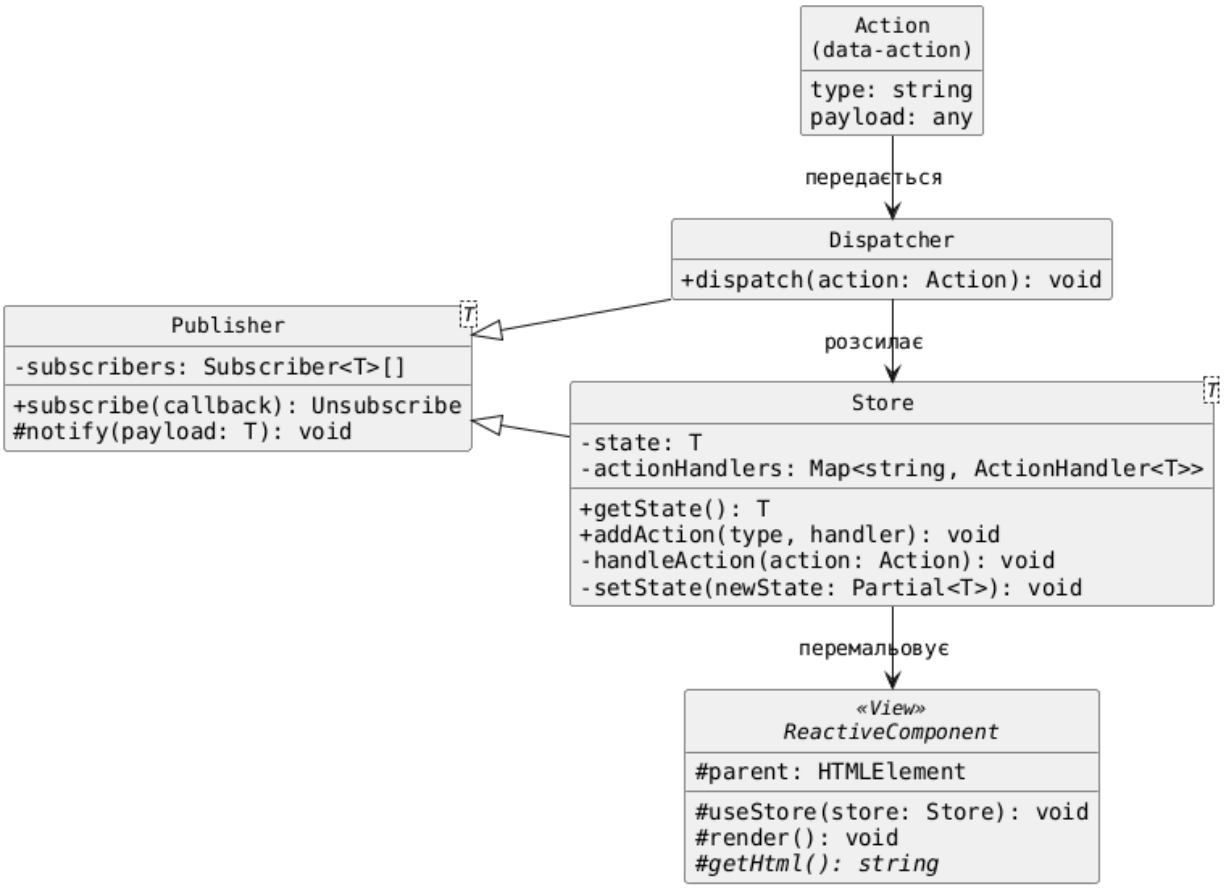

Рис. 1.1 - Діаграма основних класів Flux-подібної архітектури додатку

Взаємодія між рівнем даних і відображенням

Архітектуру окремої сторінки застосунку продемонстровано на прикладі екрану «My Posts». Точкою входу є клас MyPostsScreen, що розширює ReactiveComponent і підписується на зміни стану за допомогою useStore().

Під час взаємодії користувача (створення коментаря, редагування чи видалення допису) компонент формує об'єкт Action із відповідним типом (CREATE_COMMENT, EDIT_POST, DELETE_POST) і передає його до диспетчера.

Диспетчер направляє дію до myPostsStore, де зареєстрований обробник звертається до рівня даних – класів PostsRepository та CommentsRepository, які абстрагують CRUD-операції над localStorage.

Після оновлення даних стан сховища змінюється, що автоматично перемальовує MyPostsScreen разом із вкладеними чистими компонентами (Header, Footer, PostList). Чисті компоненти не мають власного стану і слугують виключно для побудови HTML-розмітки на основі переданих параметрів.

Така багатошарова структура – відображення, диспетчеризація, сховище, репозиторії – забезпечує чітке розмежування відповідальностей між рівнями застосунку.

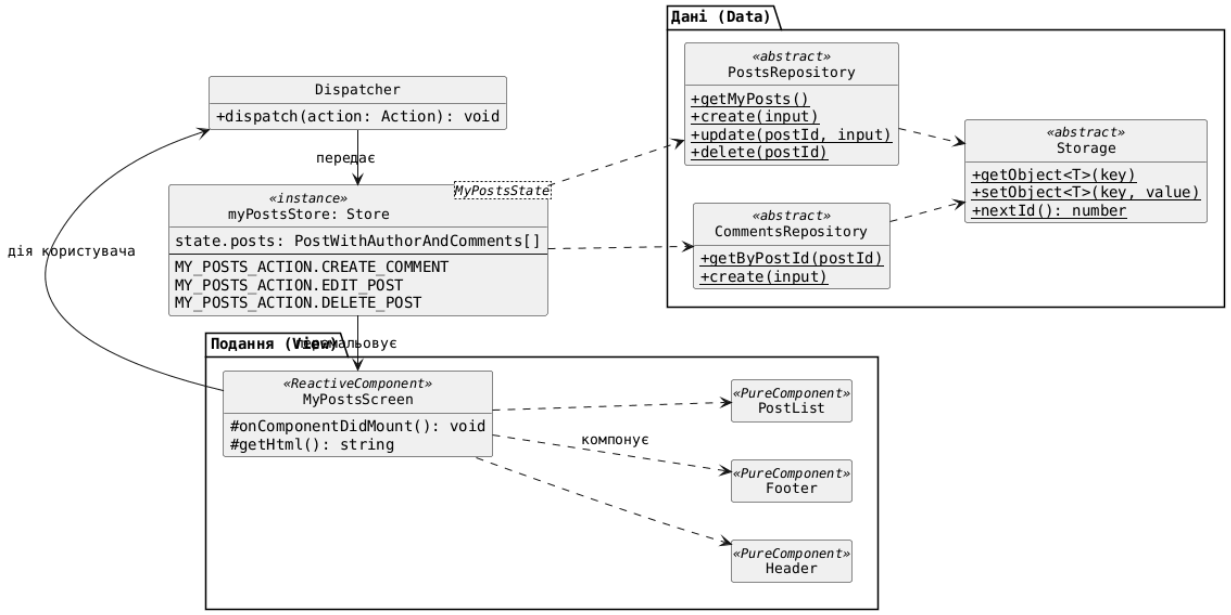

Рис. 2.1 - Діаграма взаємодії між рівнями застосунку

Скріншоти додатка

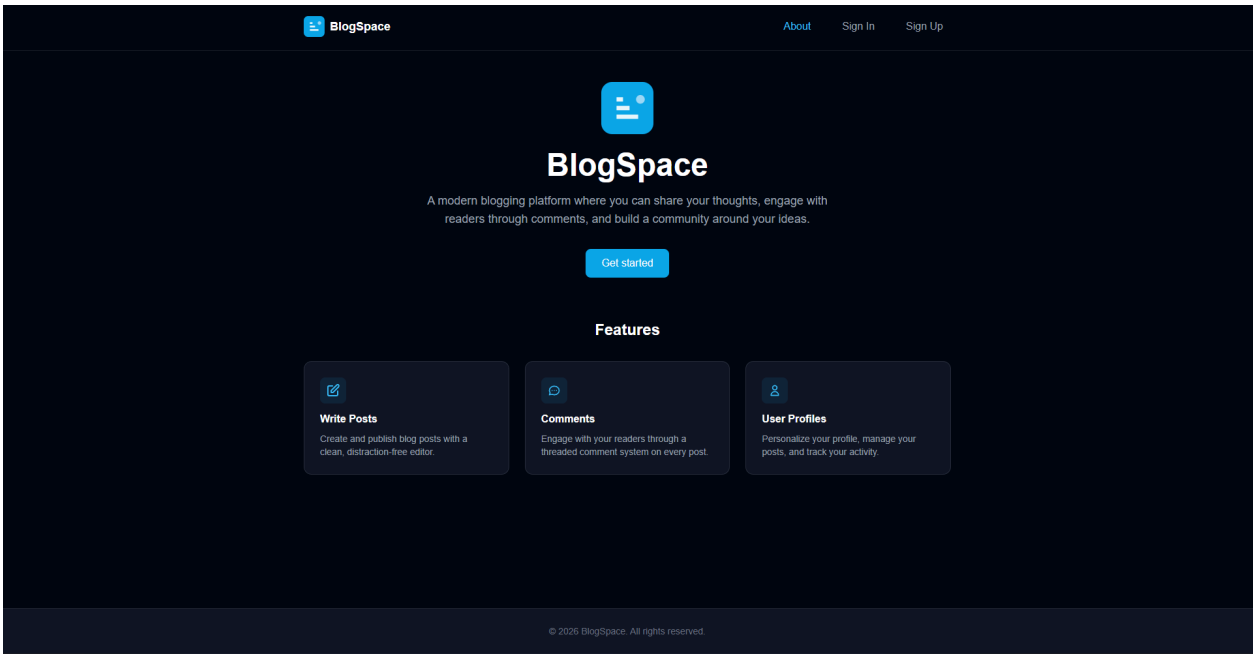

Рис. 3.1 - сторінка About (для неавторизованих користувачів)

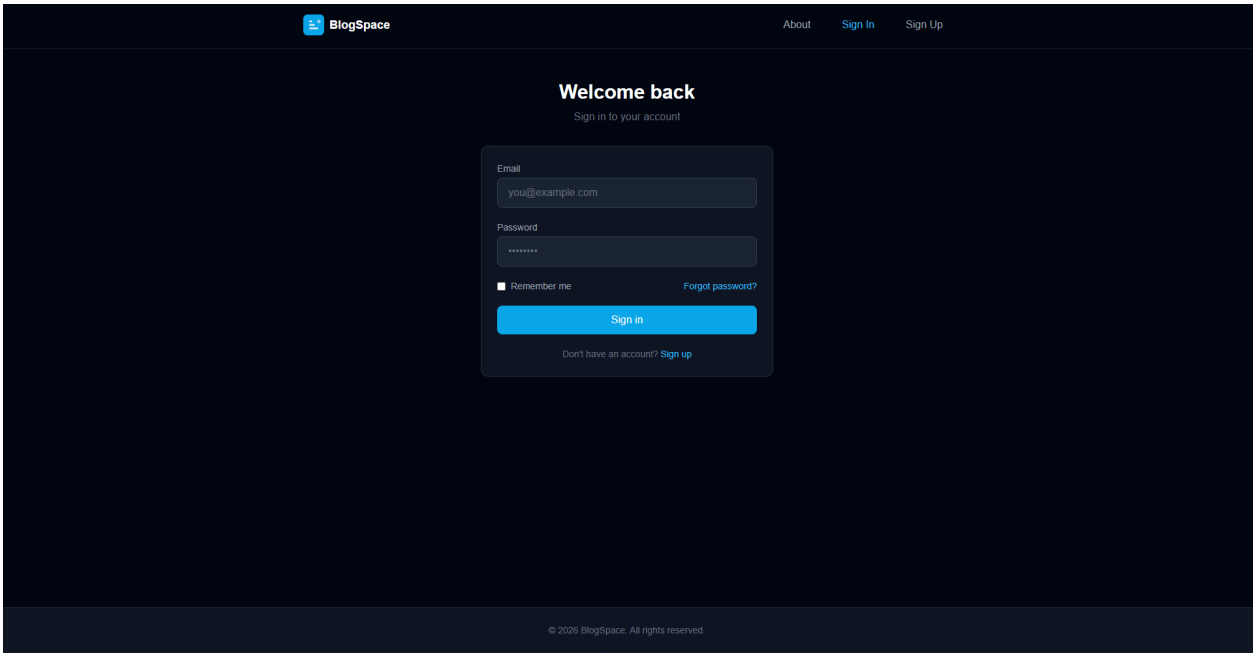

Рис. 3.2 - сторінка Sign In

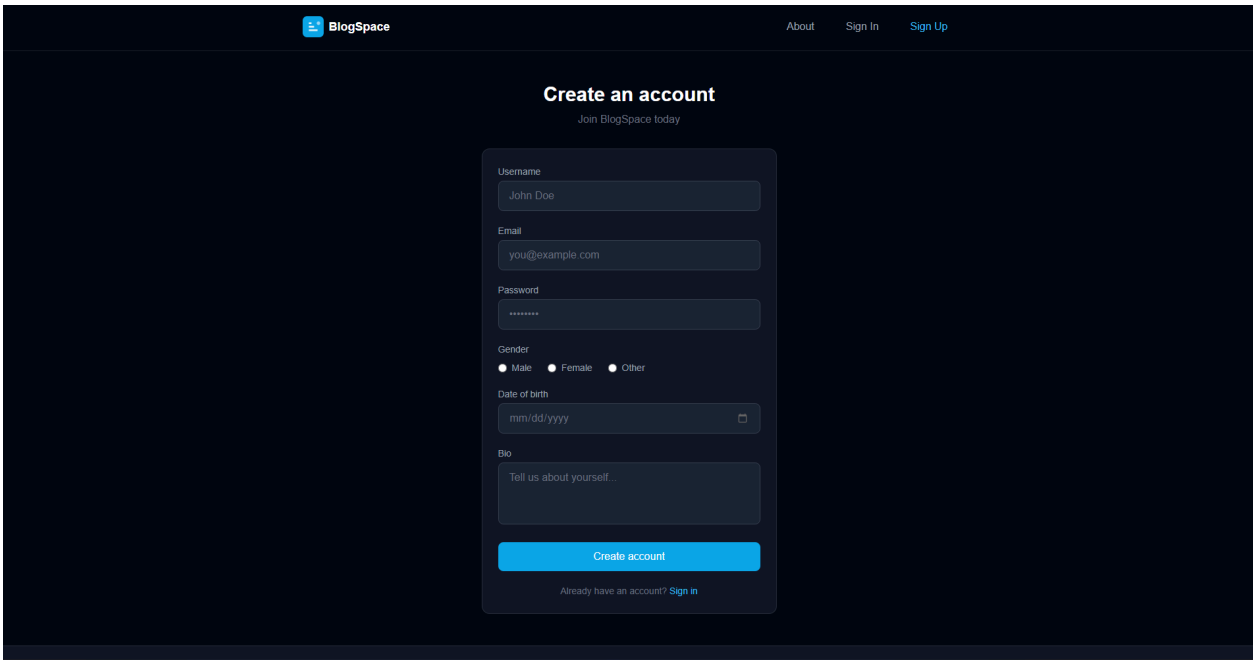

Рис. 3.3 - сторінка Sign Up

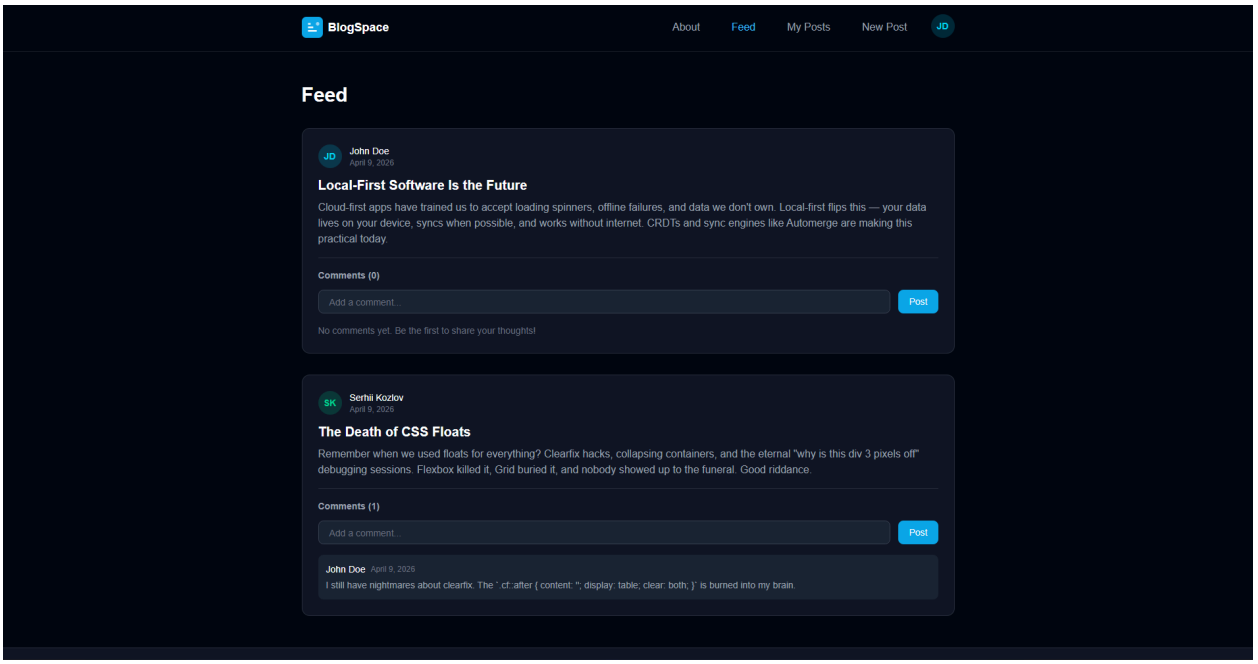

Рис. 3.4 - сторінка Feed

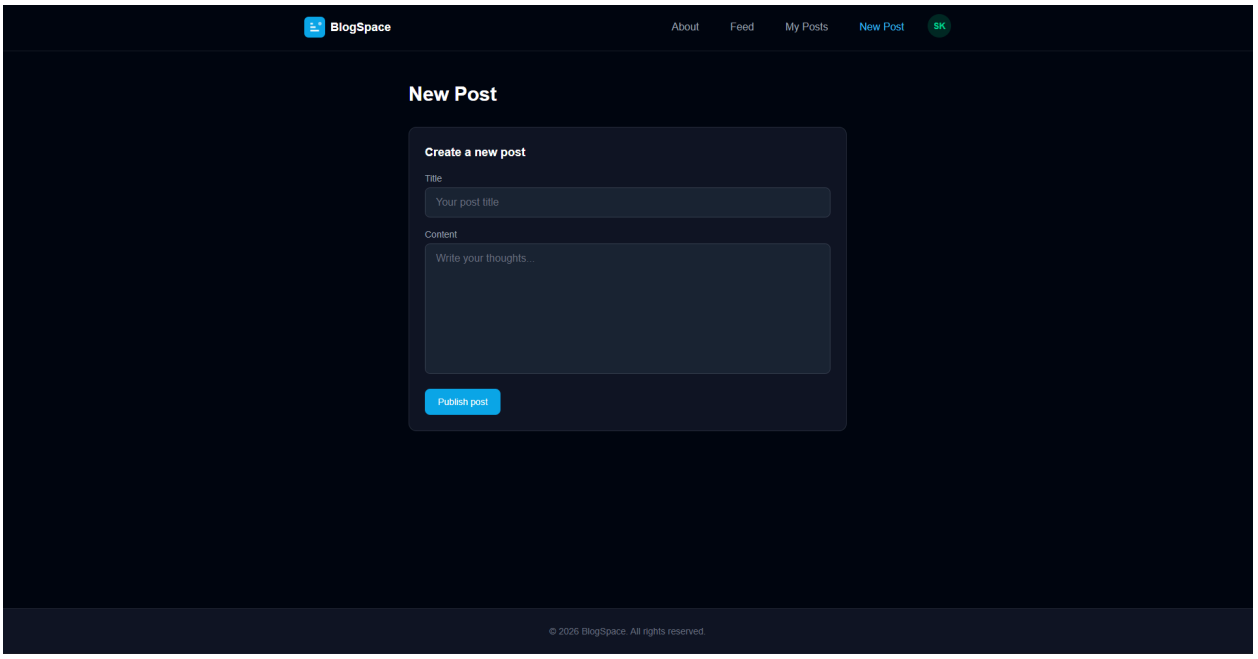

Рис. 3.5 - сторінка New Post

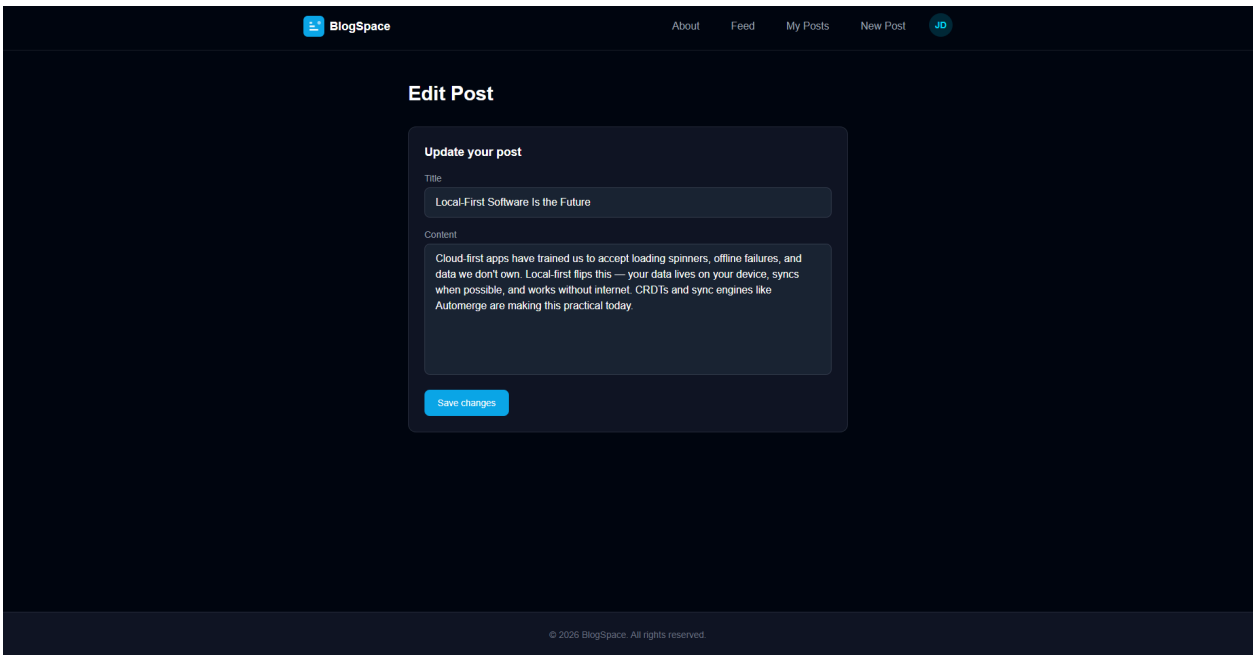

Рис. 3.6 - сторінка Edit Post

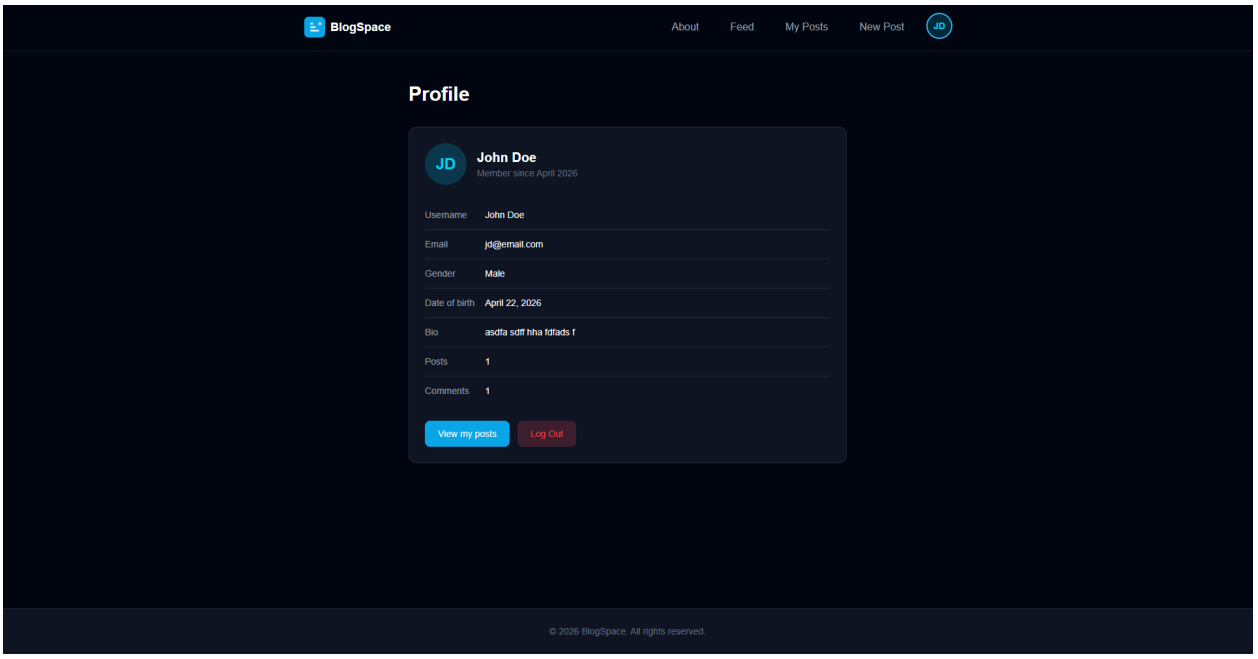

Рис. 3.7 - сторінка Profile

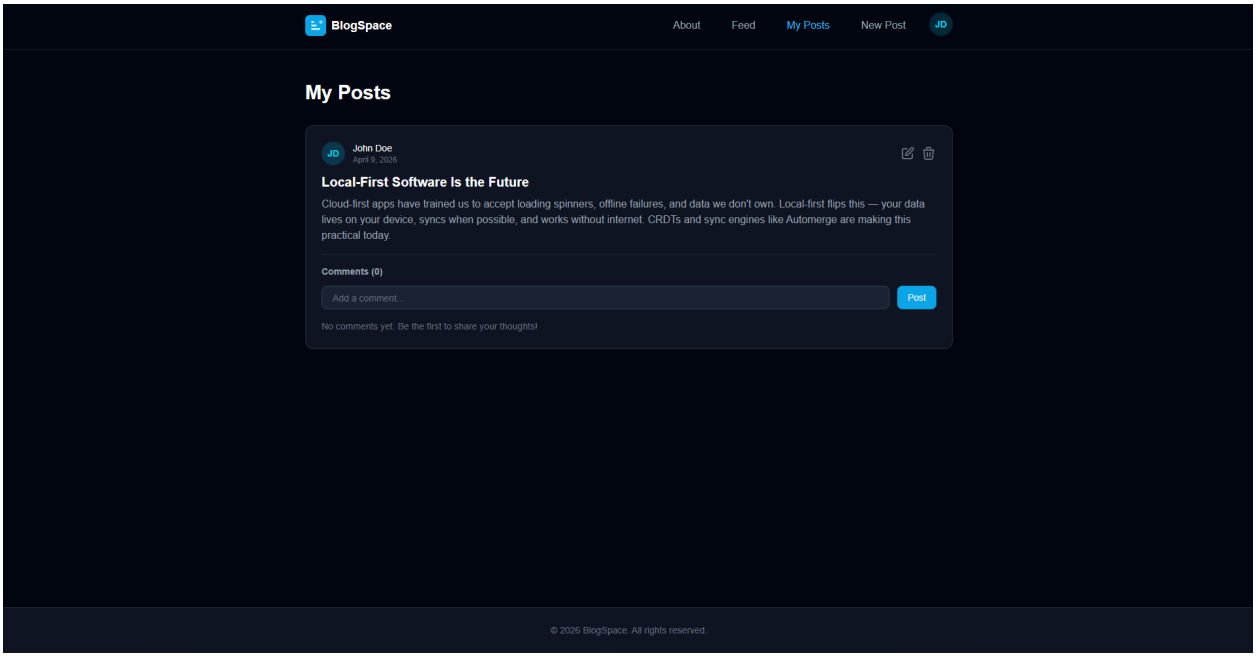

Рис. 3.8 - сторінка My Posts

Компонент сторінки My Posts

Файл: src/modules/posts/view/screens/MyPostsScreen.ts

```
import { ReactiveComponent } from "@/modules/core/component";
import { createScreen } from "@/modules/core/screen";
import { Header } from "@/modules/UI/Header";
import { Footer } from "@/modules/UI/Footer";
import { PostList } from "@/modules/posts/view/components/PostList";
import { MyPostsEmptyState } from
"@/modules/posts/view/components/MyPostsEmptyState";
import {
  myPostsStore,
  MY_POSTS_ACTION,
} from "@/modules/posts/data/myPosts.store";
import { authStore } from "@/modules/auth/data/auth.store";
import { mobileNavMenuStore } from "@/modules/UI/mobileNavMenu.store";

export class MyPostsScreen extends ReactiveComponent {
  protected onComponentDidMount(): void {
    this.useStore(myPostsStore);
    this.useStore(authStore);
    this.useStore(mobileNavMenuStore);
  }

  protected getHtml(): string {
    const { posts } = myPostsStore.getState();

    return /*html*/ `
      ${Header()}

      <main class="mx-auto w-full max-w-5xl flex-1 px-4 py-12">
        ${PostList({
          title: "My Posts",
          posts,
          EmptyState: MyPostsEmptyState,
          actions: {
            createComment: MY_POSTS_ACTION.CREATE_COMMENT,
            editPost: MY_POSTS_ACTION.EDIT_POST,
            deletePost: MY_POSTS_ACTION.DELETE_POST,
          },
        })}
      </main>

      ${Footer()}
    `;
  }
}

createScreen("root", MyPostsScreen);
```